

Desafio para candidatos — Full Stack .NET / Angular

Versão revisada (v4) — CRUD de Pedidos

Objetivo

O desafio consiste em criar uma API em .NET, desenvolvendo o CRUD de Pedidos conforme a modelagem de dados fornecida. A base de dados utilizada deve ser o SQL Server (ou InMemory / via Container, ver seção de ambiente).

É necessário implementar, no mínimo, testes unitários para o método GET de Pedidos, abrangendo tanto a camada de visualização (controller) quanto a camada de serviço.

Escopo do CRUD

Mandatário: Criar (POST) e Consultar (GET) Pedido.

Desejável: Atualizar (PUT) e Remover (DELETE) Pedido — implemente como quiser demonstrar domínio completo do CRUD.

Modelagem de dados

Modelagem de dados mínima que deve ser refletida no CRUD (chaves e tipos abaixo):

Pedido	
PK	Id int identity
	NomeCliente varchar(60)
	EmailCliente varchar(60)
	DataCriacao datetime
	Pago bit

ItensPedido	
PK	Id int identity
FK PK	IdPedido int
FK PK	IdProduto int
	Quantidade int

Produto	
PK	Id int identity
	NomeProduto varchar(20)
	Valor decimal(10,2)

Arquitetura

A API deverá ser desenvolvida em .NET, utilizando a arquitetura de sua escolha, sendo a Clean Architecture preferida. É fundamental aplicar princípios como Domain-Driven Design (DDD), SOLID, Clean Code e garantir a conformidade com as melhores práticas de APIs REST. Espera-se um código bem estruturado, de fácil manutenção e extensível, com foco em modularidade e reutilização.

O front-end pode ser desenvolvido utilizando qualquer framework ou biblioteca JavaScript de qualquer versão, porém temos preferência por Angular.

Entregáveis

Obrigatório

- Projeto compilando com sucesso;
- Conceitos de DDD, SOLID e Clean Code aplicados;
- API .NET com Criar (POST) e Consultar (GET) Pedido;
- Teste unitário back-end no mínimo do GET Pedido (controller e serviço);
- Migration SQL funcionando no projeto;
- Swagger implementado e funcionando;
- O GET de Pedido deve retornar o modelo de JSON (JSON 1);
- Tratamento padronizado de erros e validação de entrada (ex.: 400 para payload inválido, 404 para pedido inexistente), seguindo os códigos HTTP adequados
- Projeto publicado em repositório GIT.

Desejável

- CRUD completo de Pedido (Update e Delete)
 - Arquivo README com informações sobre a aplicação e decisões de arquitetura tomadas;
 - Teste unitário também para o método de criação (POST) de Pedido
 - Paginação e/ou filtros na listagem de Pedidos
 - Front-end com todas as integrações e os comportamentos básicos funcionando;
 - API e Front-end publicados em Cloud;
 - Banco de dados no Docker Compose;
 - Design Patterns.
-

Modelo de resposta — GET Pedidos

O campo `valorTotal` representa a soma dos produtos incluídos no pedido, considerando a quantidade comprada. O campo `valorUnitario` de cada item deve refletir o valor praticado naquele pedido — atente-se a isso ao decidir como modelar o dado (a tabela `ItensPedido` fornecida não possui, por padrão, uma coluna de preço).

```
{
  "id": 0,
  "nomeCliente": "",
  "emailCliente": "",
  "pago": true,
  "valorTotal": 0.0,
  "itensPedido": [
    {
      "id": 0,
      "idProduto": 0,
      "nomeProduto": "",
      "valorUnitario": 0.0,
      "quantidade": 0
    }
  ]
}
```

JSON 1 — Modelo GET Pedidos

Requisitos do ambiente

- Visual Studio 2022 ou Visual Studio Code;
- Banco de dados SQL, InMemory (detalhes abaixo) ou via Container (SQL Server). Pode-se utilizar a lib do Entity Framework Core de banco de dados em memória oficial Microsoft.

Mais informações: learn.microsoft.com — EF Core InMemory Provider